

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



MINI PROJECT

TỐI ƯU LẬP KẾ HOẠCH

**Thiết kế hệ giải đa tầng
cho bài toán xếp thời khóa biểu**

Nhóm: 13

Mã lớp học: 168497

Giảng viên hướng dẫn: Nguyễn Văn Sơn

Danh sách sinh viên thực hiện:

STT	Họ tên	Mã sinh viên
1	Kim Anh Quân	20232249
2	Đỗ Xuân Hoàng	20235084
3	Nguyễn Văn Phú Thái	20235214
4	Lâm Huy Vũ	20225117

Hà Nội, ngày 8 tháng 6 năm 2026

Bảng phân chia công việc trong nhóm

Họ và tên	MSSV	Nhiệm vụ đảm nhiệm trong dự án
Kim Anh Quân (Leader)	20232249	- Thiết kế kiến trúc tổng thể (pipeline đa tầng), điều phối tiến độ và phân công nhiệm vụ cho các thành viên. - Nghiên cứu, cài đặt và tinh chỉnh metaheuristic ALNS (phá hủy – tái tạo thích nghi, Simulated Annealing). - Thiết kế và thực thi quy trình thực nghiệm benchmark theo giao thức đánh giá công bằng. - Tổng hợp, biên tập báo cáo và xây dựng slide thuyết trình.
Nguyễn Văn Phú Thái	20235214	- Nghiên cứu và cài đặt thuật toán khởi tạo Greedy cùng tối ưu cục bộ Local Search (Ejection Chain). - Thu thập, chuyển đổi và sinh bộ dữ liệu thử nghiệm cho hệ thống.
Đỗ Xuân Hoàng	20235084	- Đồng nghiên cứu, cài đặt bộ giải chính xác CP-SAT (Interval + NoOverlap, OR-Tools). - Tích hợp warm-start từ Greedy; đánh giá giới hạn co giãn của phương pháp chính xác.
Lâm Huy Vũ	20225117	

Mục lục

Bảng phân chia công việc trong nhóm	1
Tóm tắt báo cáo (Abstract)	3
1 Giới thiệu	3
1.1 Bài toán và động lực	3
1.2 Kiến trúc hệ thống (Đa tầng: Exact $\rightarrow$ Heuristic $\rightarrow$ Metaheuristic)	4
1.3 Hàm mục tiêu và ràng buộc	4
2 Cấu trúc dữ liệu và quản lý trạng thái (State Representation)	5
2.1 Biểu diễn đối tượng (ClassDTO, RoomDTO)	5
2.2 Ma trận trạng thái thời gian (room_busy, teacher_busy)	5
2.3 Đánh giá hợp lệ cục bộ (Fast Feasibility Check)	5
3 Chiến lược khởi tạo lời giải ban đầu	6
3.1 Sắp xếp ưu tiên	6
3.2 Greedy: First-feasible insertion	6
3.3 Lắp gán đến điểm dừng	6
4 Lời giải chuẩn xác (Exact Method) với OR-Tools CP-SAT	7
4.1 Mô hình toán học: từ quy hoạch nguyên sang quy hoạch ràng buộc	7
4.2 Hiện thực hóa bằng Interval Variables và AddNoOverlap	7
4.3 Tăng tốc bằng Warm-start từ nghiệm Greedy	7
4.4 Giới hạn co giãn: bức tường tổ hợp	7
5 Local Search (LS) với cơ chế Ejection Chain	9
5.1 Tư duy thuật toán: phá vỡ bế tắc của Greedy	9
5.2 Cơ chế chuỗi thay thế (Giải phóng – Chèn – Tái định vị)	9
5.3 Đánh giá chênh lệch và điều kiện chấp nhận (First Improvement)	10
5.4 Giới hạn của thuật toán (Hiệu ứng Plateau)	10
6 Thuật toán Adaptive Large Neighborhood Search (ALNS)	10
6.1 Ba toán tử Phá hủy (Random, Worst-Time, Related-Teacher)	10
6.2 Ba toán tử Sửa chữa (Greedy, Regret-2, First-Possible)	10
6.3 Cập nhật trọng số Adaptive Roulette-Wheel (π_1, π_2, π_3)	11
6.4 Tiêu chí chấp nhận bằng Luyện kim mô phỏng (Simulated Annealing)	11
7 Phân tích độ nhạy và tinh chỉnh tham số (Parameter Tuning)	12
7.1 Sự tiến hóa của xác suất chọn toán tử (cân bằng Khám phá – Khai thác)	12
7.2 Tác động của hệ số học λ (Learning Rate)	13
7.3 Sự biến thiên Nhiệt độ T và hệ số làm mát α (Cooling Schedule)	13
8 Kết quả thực nghiệm	14
8.1 Thiết lập đánh giá công bằng (Môi trường, Time-limit, Seeds)	14
8.2 Bảng kết quả đầy đủ (phân nhóm: Nhỏ, Trung bình, Lớn)	14
8.3 Biểu đồ so sánh (chất lượng nghiệm và thời gian chạy)	15
8.4 Phân tích và thảo luận	16
9 Kết luận	17
Tài liệu tham khảo	18

Tóm tắt báo cáo (Abstract)

Luận điểm thiết kế. Bài toán xếp thời khóa biểu là NP-khó: không tồn tại một thuật toán đơn lẻ vừa *đảm bảo tối ưu*, vừa *phản hồi tức thời*, vừa *co giãn* tới quy mô toàn trường. Thay vì cố ép một thuật toán làm mọi việc, báo cáo này *thiết kế một hệ giải đa tầng* và lập luận cho từng quyết định kiến trúc bằng số liệu thực nghiệm.

Phát hiện dẫn dắt. Qua đo đạc, chúng tôi chỉ ra nguồn độ khó thực sự *không phải kích thước N* mà là *độ chặt tài nguyên*: bộ giải chính xác CP-SAT giải tối ưu dữ liệu “thưa” tới tận $N = 600$, nhưng trên dữ liệu thực “nghèn” tài nguyên chỉ đạt nghiệm khả thi (còn khoảng cách cận) ngay từ $N = 200$. Phát hiện này quyết định cách định tuyến công cụ.

Kiến trúc đề xuất. Một bộ định tuyến đẩy bài toán vào một trong ba tầng, tất cả cùng dùng nghiệm Tham lam (Greedy) làm hạt giống:

1. **Tầng chính xác (CP-SAT):** cho nghiệm tối ưu có chứng nhận ở quy mô nhỏ — đóng vai trò *thước đo vàng*.
2. **Tầng tức thời (Local Search):** cải thiện cục bộ trong mili-giây cho ứng dụng tương tác.
3. **Tầng co giãn (ALNS):** phá hủy–tái tạo thích nghi để khai phá toàn cục cho hệ thống lớn.

Cam kết trung thực. Mọi hệ số (T, α, λ) đều được biện minh bằng đồ thị độ nhạy; benchmark được chạy theo *giao thức công bằng* (cùng ngân sách thời gian, seed cố định); toàn bộ số liệu sinh tự động từ chính mã nguồn được phân tích, đảm bảo tái lập.

1 Giới thiệu

1.1 Bài toán và động lực

Bài toán xếp thời khóa biểu thuộc lớp *phân bổ tài nguyên thời gian/phòng học*: gán cho mỗi lớp một phòng và một tiết bắt đầu sao cho thỏa các ràng buộc tài nguyên và tối đa hóa số lớp được xếp. Đây là một biến thể *gán có giới hạn tài nguyên* (resource-constrained assignment) — NP-khó: các ràng buộc chống xung đột làm không gian nghiệm khả thi vỡ vụn, và mỗi lớp có tới hàng trăm lựa chọn vị trí.

Động lực thực tế: cùng một bài toán nhưng xuất hiện ở nhiều quy mô và yêu cầu trái ngược nhau — từ xếp lịch một bộ môn (vài chục lớp, cần tối ưu tuyệt đối), tới công cụ chỉnh lịch tương tác (cần phản hồi tức thời), tới xếp lịch toàn trường (hàng nghìn lớp, chạy theo lô). Một phát hiện quan trọng dẫn dắt toàn bộ thiết kế: *nguồn độ khó không phải kích thước N mà là độ chặt tài nguyên* — điều được kiểm chứng định lượng ở mục 4.4.

Hình 1 minh họa bài toán: gán mỗi lớp vào một khối t_i tiết *liên tiếp* trong một phòng, tôn trọng ba ràng buộc cứng (sức chứa, không trùng phòng, không trùng giáo viên) và không vắt chéo ngày.

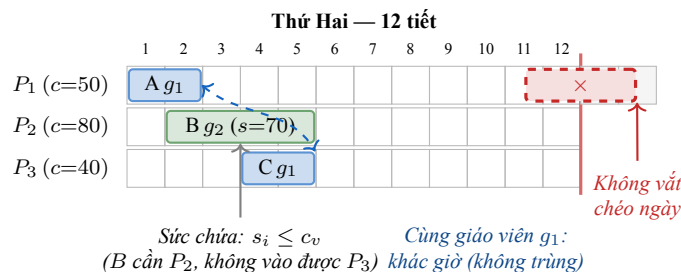


Figure 1: Minh họa bài toán trên một ngày (tuần có 5 ngày $\times$ 12 tiết = 60 tiết). Mỗi lớp là một khối t_i tiết liên tiếp đặt trong một phòng; màu khối = giáo viên. Ba ràng buộc cứng: **sức chứa** ($s_i \leq c_v$), **không trùng phòng** (mỗi ô tối đa một lớp), **không trùng giáo viên**; khối đỏ vì phạm *vắt chéo ngày*. Mục tiêu: tối đa hóa số lớp xếp được.

1.2 Kiến trúc hệ thống (Đa tầng: Exact → Heuristic → Metaheuristic)

Vì không một thuật toán nào tối ưu cho mọi quy mô, hệ thống được tổ chức thành *ba tầng* với một bộ định tuyến phân loại theo quy mô/độ chặt (Hình 2). Điểm thiết kế mấu chốt: cả ba tầng dùng chung nghiệm **Greedy** làm hạt giống (warm-start cho CP-SAT, cấu hình nền cho Local Search và ALNS).

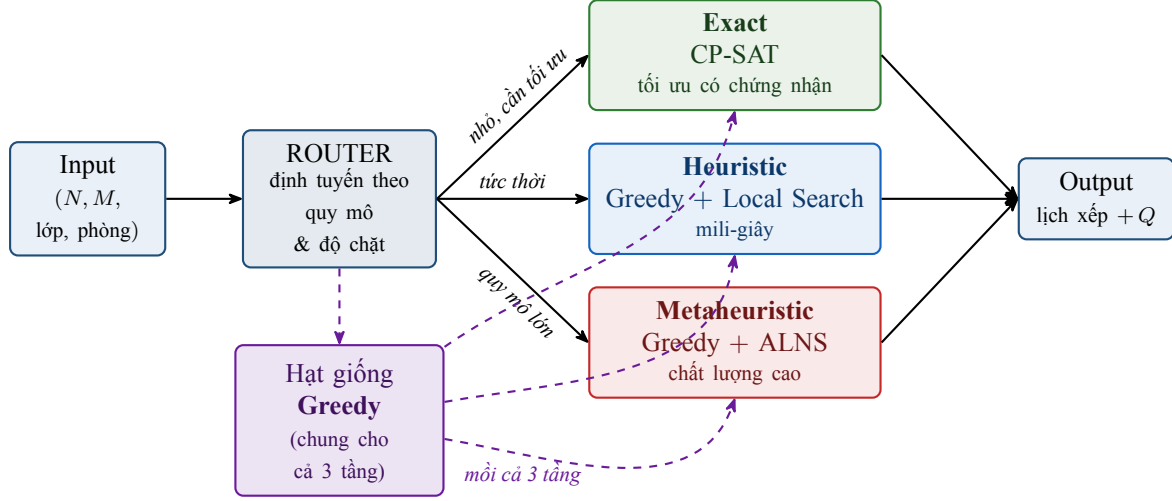


Figure 2: Kiến trúc đa tầng *Exact → Heuristic → Metaheuristic*. Bộ Router định tuyến theo quy mô và độ chặt tài nguyên; nghiệm Greedy (nét đứt tím) là hạt giống chung cho cả ba tầng.

Báo cáo đi theo đúng luồng dữ liệu của hệ thống: cấu trúc dữ liệu & trạng thái (Mục 2) → khởi tạo (Mục 3) → ba tầng giải (Mục 4, 5, 6) → tinh chỉnh tham số (Mục 7) → thực nghiệm (Mục 8).

1.3 Hàm mục tiêu và ràng buộc

Cấu trúc thời gian: $D = 5 \text{ ngày} \times H = 12 \text{ tiết}$, cho không gian $\mathcal{T} = \{0, \dots, 59\}$. Đầu vào: tập lớp $\mathcal{C} = \{1, \dots, N\}$ với mỗi lớp i có (t_i, g_i, s_i) (thời lượng, giáo viên, sĩ số); tập phòng $\mathcal{R} = \{1, \dots, M\}$ với sức chứa c_r .

Miền vị trí hợp lệ. Vì lớp không vắt chéo ngày, tập tiết bắt đầu hợp lệ của lớp thời lượng t_i là

$$\mathcal{S}(t_i) = \{d \cdot H + o \mid d \in \{0, \dots, D-1\}, o \in \{0, \dots, H-t_i\}\}. \quad (1)$$

Ràng buộc *liên tục* và *không xuyên ngày* được hấp thụ trọn vẹn vào miền này — không cần bất đẳng thức tường minh.

Biến quyết định, ràng buộc, mục tiêu. Đặt $x_{i,r,u} = 1$ nếu lớp i xếp vào phòng r bắt đầu tại $u \in \mathcal{S}(t_i)$, và $a_i = \sum_{r,u} x_{i,r,u}$.

$$\sum_r \sum_{u \in \mathcal{S}(t_i)} x_{i,r,u} = a_i \leq 1 \quad \forall i \quad (\text{mỗi lớp tối đa một vị trí}), \quad (2)$$

$$x_{i,r,u} = 0 \text{ nếu } c_r < s_i \quad (\text{sức chứa}), \quad (3)$$

$$\sum_i \sum_{u \leq \tau < u+t_i} x_{i,r,u} \leq 1 \quad \forall r, \tau \quad (\text{xung đột phòng}), \quad (4)$$

$$\sum_{i: g_i = g} \sum_r \sum_{u \leq \tau < u+t_i} x_{i,r,u} \leq 1 \quad \forall g, \tau \quad (\text{xung đột giáo viên}). \quad (5)$$

$$\boxed{\max Q = \sum_{i \in \mathcal{C}} a_i.} \quad (6)$$

2 Cấu trúc dữ liệu và quản lý trạng thái (State Representation)

Hiệu năng của cả ba tầng phụ thuộc vào việc *kiểm tra hợp lệ* và *cập nhật* một phép gán phải thật nhanh, vì các thuật toán xấp xỉ thực hiện hàng triệu thao tác này. Mục này trình bày biểu diễn dữ liệu được thiết kế cho mục tiêu đó.

2.1 Biểu diễn đối tượng (ClassDTO, RoomDTO)

Hai đối tượng dữ liệu thuần (DTO) tách *thuộc tính đầu vào* khỏi *trạng thái lời giải*:

- ClassDTO: bất biến (t_i, g_i, s_i) cộng phần trạng thái thay đổi theo lời giải — `assigned_slot` (tiết bắt đầu, 1-based), `assigned_room`, `is_assigned`. Nhờ tách bạch này, mọi thuật toán dùng *chung* một biểu diễn lớp; chỉ phần trạng thái bị ghi đè khi gán/gỡ.
- RoomDTO: định danh phòng và sức chứa c_v .

Các mảng `classes[]`, `rooms[]` dùng chỉ số *1-based* (phần tử 0 để trống) để khớp định danh lớp/phòng trong đề và định dạng xuất.

2.2 Ma trận trạng thái thời gian (`room_busy`, `teacher_busy`)

Trái tim của lớp `ScheduleState` là hai ma trận bận theo thời gian:

<code>room_busy[r][τ]</code>	phòng r có bị chiếm tại tiết τ không
<code>teacher_busy[g][τ]</code>	giáo viên g có bận tại tiết τ không

Đây là hiện thực trực tiếp của hai họ ràng buộc xung đột (4)–(5): một phép gán hợp lệ tương đương với việc cả khối t_i tiết của lớp đều *rảnh* trên cả hai ma trận. `teacher_busy` dùng dict (chỉ sinh hàng cho giáo viên thực sự xuất hiện) để tiết kiệm bộ nhớ khi mã giáo viên thừa.

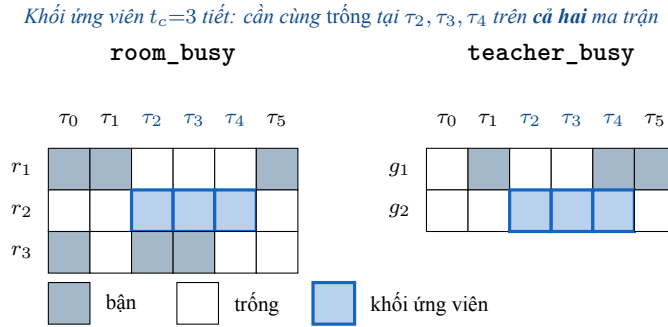


Figure 3: Kiểm tra tính hợp lệ: để xếp một lớp $t_c=3$ tiết vào phòng r_2 do giáo viên g_2 dạy, hệ thống chỉ cần tìm một khối 3 ô *cùng trống* ở cùng các tiết (τ_2, τ_3, τ_4) trên `room_busy[r2]` và `teacher_busy[g2]`.

Chèn / gỡ trong $\mathcal{O}(t_i)$. Thao tác `insert` đánh dấu t_i ô bận trên hai ma trận và ghi trạng thái vào ClassDTO; `remove` làm ngược lại. Cả hai chỉ tốn $\mathcal{O}(t_i) \leq \mathcal{O}(4)$ — gần như hằng số. Ngoài ra `ScheduleState` cung cấp `snapshot/restore` nhẹ (chỉ lưu ánh xạ `class_id` → (phòng, tiết)) để metaheuristic có thể thử một nước đi rồi hoàn tác tức thì — nền tảng cho Local Search và ALNS.

2.3 Đánh giá hợp lệ cục bộ (Fast Feasibility Check)

Hàm `can_place(c, r, s0)` trả lời “lớp c có đặt được vào phòng r từ tiết s_0 không” theo đúng thứ tự cắt tia từ rẻ tới đắt:

1. Kiểm tra sức chứa $c_r \geq s_c$ (một phép so sánh) — loại nhanh phần lớn phòng không hợp lệ.

2. Quét t_c ô trên `room_busy` và `teacher_busy`; gặp ô bận đầu tiên là trả False ngay (short-circuit).

Cùng nhóm tiện ích: `find_position` (vị trí trống đầu tiên cho một lớp) và `feasible_room_count` (đếm số phòng khả thi — dùng cho Regret-2 ở Mục 6). Tập $\mathcal{S}(t_i)$ được *cache* theo thời lượng để không phải sinh lại. Nhờ thiết kế này, một lần kiểm tra hợp lệ chỉ tốn $\mathcal{O}(t_i)$, cho phép các vòng lặp tìm kiếm chạy hàng chục nghìn lần/giây.

3 Chiến lược khởi tạo lời giải ban đầu

Greedy tạo nghiệm khởi đầu *hạt giống* dùng chung cho cả ba tầng. Mục tiêu: nhanh, hợp lệ, và đủ tốt để các tầng sau chỉ cần cải thiện.

3.1 Sắp xếp ưu tiên

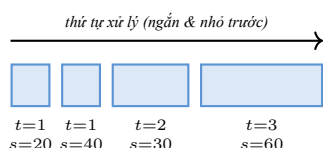
Thứ tự xử lý quyết định chất lượng Greedy, theo hai tiêu chí:

- **Thứ tự lớp:** sắp theo *thời lượng t tăng dần*, rồi *sĩ số s tăng dần* — ưu tiên lớp *ngắn* và *ít sinh viên* trước. Lớp ngắn linh hoạt, dễ lấp vào các khe thời gian nhỏ, nên xếp sớm giúp giữ lại các khối thời gian dài liền mạch cho những lớp khó hơn về sau.
- **Thứ tự phòng (best-fit):** duyệt phòng theo *sức chứa tăng dần* — luôn thử phòng *vừa đủ* chứa trước, để dành các phòng lớn cho những lớp đông sinh viên thực sự cần đến chúng, tránh lãng phí tài nguyên khan hiếm.

3.2 Greedy: First-feasible insertion

Với mỗi lớp theo thứ tự ưu tiên, duyệt tiết bắt đầu hợp lệ $s_0 \in \mathcal{S}(t_i)$ (vòng ngoài) rồi phòng theo sức chứa tăng dần (vòng trong); kiểm tra giáo viên rồi phòng có rảnh cả khối t_i tiết không, và lấy cặp (tiết, phòng) hợp lệ *đầu tiên* để gán luôn. Chiến lược “first-feasible” (thay vì tìm vị trí tối ưu) giữ chi phí ở mức thấp nhất — đây là lý do Greedy chạy trong *micro-giây* ngay cả với $N = 1000$.

1. Sắp ưu tiên: $t \uparrow$, rồi $s \uparrow$



2. Best-fit: phòng vừa đủ chứa trước

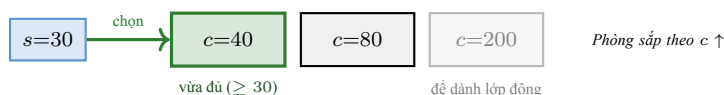


Figure 4: Chiến lược khởi tạo Greedy. **(1)** Lớp được xử lý theo thứ tự *thời lượng tăng*, *sĩ số tăng* (ngắn & ít SV trước). **(2)** Với mỗi lớp, quét tiết bắt đầu (vòng ngoài) rồi phòng theo *sức chứa tăng dần* và chọn phòng *vừa đủ* chứa đầu tiên (best-fit, first-feasible) — để dành phòng lớn cho lớp đông. Quá trình lặp lại đến khi không xếp thêm được lớp nào.

3.3 Lặp gán đến điểm dừng

Greedy không gán một lần rồi dừng: nó *lặp lại* các lượt quét cho đến khi một lượt không xếp thêm được lớp nào (điểm dừng). Cơ chế lặp này gắn liền với Local Search ở Mục 5: sau mỗi lần Ejection Chain tái sắp xếp và giải phóng chỗ, một lượt Greedy mới có thể tận dụng để nhồi thêm lớp.

Nhận xét 1. *Greedy không cố tối ưu — nó cố nhanh và hợp lệ. Chính vì còn dư địa cải thiện (đặc biệt trên dữ liệu ngẫu nhiên), ba tầng phía sau mới có việc để làm; và vì cả ba cùng xuất phát từ một mốc Greedy, mọi so sánh ở Mục 8 đều quy về cùng một điểm tham chiếu.*

4 Lời giải chuẩn xác (Exact Method) với OR-Tools CP-SAT

Tầng Exact cung cấp nghiệm tối ưu *có chứng nhận* ở quy mô nhỏ — vừa là lời giải cho dữ liệu khoa/viện, vừa là *thước đo vàng* để chấm điểm các tầng xấp xỉ (Mục 8).

4.1 Mô hình toán học: từ quy hoạch nguyên sang quy hoạch ràng buộc

Công thức (2)–(6) là một mô hình quy hoạch nguyên (ILP). Tuy nhiên, biểu diễn trực tiếp các họ ràng buộc xung đột (4)–(5) sinh ra hàng vạn bất đẳng thức (một cho mỗi cặp phòng/giáo viên $\times$ tiết) và một mô hình lỏng lẻo. Ta chuyển sang *quy hoạch ràng buộc* (CP), nơi cấu trúc thời gian được biểu diễn cô đọng và có bộ truyền lan ràng buộc (propagator) chuyên dụng.

4.2 Hiện thực hóa bằng Interval Variables và AddNoOverlap

Với mỗi cặp (lớp i , phòng r) khả thi ($c_r \geq s_i$):

- biến hiện diện $b_{i,r} \in \{0, 1\}$;
- biến bắt đầu $\text{start}_{i,r}$ ràng buộc chỉ nhận giá trị trong $\mathcal{S}(t_i)$ (qua `AddAllowedAssignments`) — nơi cài đặt ràng buộc logic;
- *Optional Interval* $\text{intv}_{i,r}$ — khối thời lượng t_i chỉ “tồn tại” khi $b_{i,r} = 1$.

Khi đó hai họ ràng buộc xung đột thu gọn thành các lời gọi *không-chồng-lấn*:

$$\text{AddNoOverlap}(\{\text{intv}_{i,r} : i\}) \quad \forall r \quad (\text{mỗi phòng}), \quad (7)$$

$$\text{AddNoOverlap}(\{\text{intv}_{i,r} : g_i = g\}) \quad \forall g \quad (\text{mỗi giáo viên}), \quad (8)$$

với $\sum_r b_{i,r} = a_i$ và mục tiêu $\text{Maximize}(\sum_i a_i)$. Cách này giảm mạnh số ràng buộc tường minh và để propagator của CP-SAT xử lý xung đột thời gian hiệu quả hơn nhiều so với ILP tương đương.

4.3 Tăng tốc bằng Warm-start từ nghiệm Greedy

Trước khi giải, ta nạp nghiệm Greedy làm gợi ý (`AddHint`, không phải ràng buộc cứng): $a_i \leftarrow 1$, $b_{i,r} \leftarrow \mathbf{1}[r=r_i^g]$, $\text{start} \leftarrow u_i^g$. Solver có ngay một cận dưới khả thi để cắt nhánh mạnh hơn, không phí thời gian dò nghiệm khả thi đầu tiên, và đảm bảo kết quả không bao giờ tệ hơn Greedy.

4.4 Giới hạn cơ giản: bức tường tổ hợp

Số biến tăng tuyến tính $\mathcal{O}(N \cdot M \cdot \max_i |\mathcal{S}(t_i)|)$, còn chi phí *chứng minh* tối ưu tăng theo hàm mũ. Trên dữ liệu *thưa*, nhờ warm-start CP-SAT vẫn giải tối ưu tới $N = 600$ (Hình 5); bức tường chỉ lộ ra trên dữ liệu *nguyên* (Bảng 2).

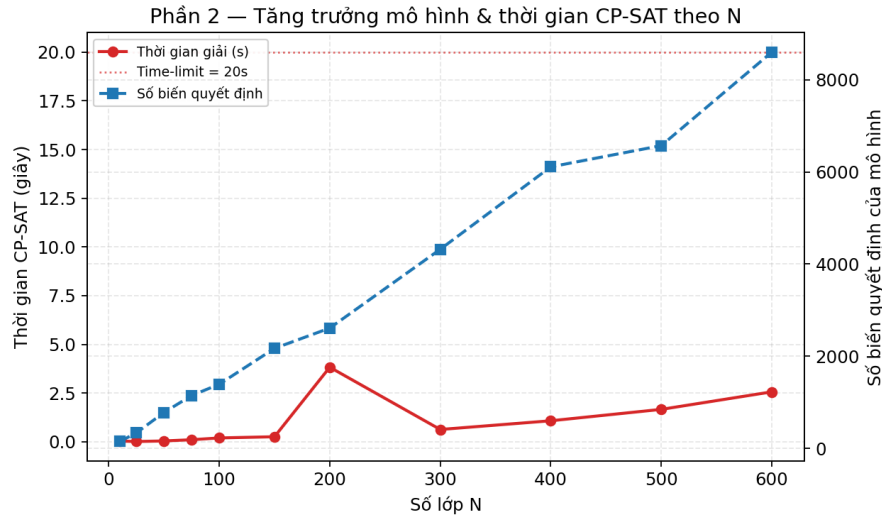


Figure 5: Dữ liệu thừa: số biến (xanh) tăng tuyến tính theo N , nhưng thời gian giải (đỏ) vẫn xa dưới time-limit. Kích thước một mình không đánh gục CP-SAT.

Table 1: CP-SAT trên dữ liệu ngẫu nhiên “loose” ($M = 8$): số biến và thời gian tăng theo N , vẫn chứng minh được tối ưu.

N	Số biến	Thời gian (s)	Trạng thái	Q (obj)	Gap
10	152	0.04	OPTIMAL	10	0
25	339	0.01	OPTIMAL	25	0
50	782	0.04	OPTIMAL	48	0
75	1149	0.10	OPTIMAL	69	0
100	1394	0.20	OPTIMAL	98	0
150	2172	0.26	OPTIMAL	146	0
200	2612	3.83	OPTIMAL	198	0
300	4322	0.63	OPTIMAL	232	0
400	6110	1.08	OPTIMAL	255	0
500	6570	1.66	OPTIMAL	294	0
600	8598	2.56	OPTIMAL	306	0

Table 2: CP-SAT trên dữ liệu thực nghiệm tài nguyên (time-limit 20s). Khi N lớn, solver chỉ đạt feasible với gap dương: không kịp chứng minh tối ưu.

Instance	N	M	Số biến	Wall (s)	Trạng thái	Greedy	Q	Cận trên	Gap
hustack01	200	10	3074	20.36	FEASIBLE	182	196	197	1
hustack02	500	50	38868	20.51	FEASIBLE	410	410	418	8
test08	1000	50	60002	20.93	FEASIBLE	959	961	1000	39

Kết quả Bảng 2 là minh chứng cho luận điểm trung tâm: đối lập với dữ liệu thừa (tối ưu tới $N=600$), trên cả ba instance thực nghiệm tài nguyên CP-SAT *không đóng được khoảng cách* trong 20s — đều dừng ở feasible với gap dương ngay cả khi đã warm-start (hustack01 $N=200$ gap 1; hustack02 $N=500$ gap

8; test08 $N=1000$ gap 39). Đây là cơ sở định lượng cho ngưỡng $N \leq 100$ của bộ định tuyến: vượt ngưỡng, nguy cơ chỉ nhận nghiệm feasible là quá cao, buộc chuyển sang Local Search (Mục 5) và ALNS (Mục 6).

5 Local Search (LS) với cơ chế Ejection Chain

Tầng Heuristic phục vụ ngữ cảnh *phản hồi nhanh*: cải thiện nghiệm Greedy thường trong mili-giây (tới vài giây trên instance lớn nghẽn), đủ nhanh cho công cụ chỉnh lịch tương tác và nhanh hơn ALNS nhiều bậc.

5.1 Tư duy thuật toán: phá vỡ bế tắc của Greedy

Greedy phụ thuộc thứ tự xử lý nên thường dễ sót những lớp bị *một* lớp khác chặn chỗ: lớp u không còn vị trí nào trống chỉ vì lớp v đang chiếm đúng khe nó cần. Local Search nhắm chính vào thể bế tắc này: thay vì chấp nhận bỏ u , nó thử *tái sắp xếp* để giải phóng chỗ.

5.2 Cơ chế chuỗi thay thế (Giải phóng – Chèn – Tái định vị)

Mỗi vòng chọn ngẫu nhiên một lớp chờ u và áp một toán tử lân cận duy nhất — *Ejection Chain* (Thuật toán 1): chèn trực tiếp nếu còn chỗ; nếu không thì thực hiện ba bước:

- **Giải phóng lớp v** : xóa bỏ lớp v đang xếp khỏi vị trí hiện tại để giải phóng không gian thời gian/phòng học;
- **Chèn lớp u** : đặt lớp u vào chỗ vừa trống (toán tử Insertion chuẩn);
- **Tái định vị lớp v** : tìm một tọa độ không gian–thời gian mới phù hợp cho lớp v vừa bị đẩy ra.

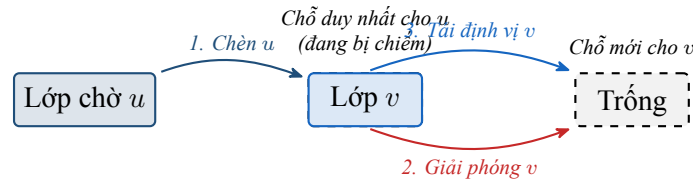


Figure 6: Cơ chế Ejection Chain: Lớp u cần chỗ, hệ thống *giải phóng* lớp v đang chiếm chỗ đó, rồi *tái định vị* v vào một chỗ trống khác.

Algorithm 1: Local Search bằng Ejection Chain

Input: nghiệm Greedy; tập lớp chưa xếp U

```

1 for mỗi bước lặp và  $U \neq \emptyset$  và chưa hết ngân sách do
2   chọn ngẫu nhiên lớp chờ  $u \in U$ ;
3   if chèn trực tiếp  $u$  được then
4     chèn  $u$ ; cập nhật nghiệm tốt nhất;
5   else
6     giải phóng một lớp  $v$  đang xếp; chèn  $u$  vào chỗ vừa trống;
7     if tìm được chỗ mới cho  $v$  then
8       tái định vị  $v$  (số lớp không đổi, cấu trúc mới);
9     else if  $t_u < t_v$  then
10      giữ  $u$  (tốn ít tiết hơn  $\Rightarrow$  lợi về sau);
11     else
12      hoàn tác: trả  $v$  về chỗ cũ;
13 return nghiệm tốt nhất đã lưu;
```

5.3 Đánh giá chênh lệch và điều kiện chấp nhận (First Improvement)

LS theo chiến lược *First Improvement*: chấp nhận ngay nước đi đầu tiên *có lợi* thay vì duyệt toàn bộ lân cận tìm nước tốt nhất — ưu tiên tốc độ. Tiêu chí “có lợi” rất gọn: nếu tái định vị được v thì giữ (Q không giảm, cấu trúc đôi, mở cơ hội mới); nếu không thì chỉ giữ u khi $t_u < t_v$ (đổi một lớp dài lấy một lớp ngắn, giải phóng tài nguyên rỗng), ngược lại hoàn tác. Nhờ luôn lưu *nghiệm tốt nhất* qua snapshot, số lớp xếp được *không bao giờ giảm*.

5.4 Giới hạn của thuật toán (Hiệu ứng Plateau)

Ejection Chain chỉ thay đổi *cục bộ* (đổi chỗ từng cặp lớp). Khi mọi hoán đổi đơn lẻ đều không mở thêm chỗ, lời giải mắc kẹt trên một *cao nguyên*: nó không thể tạo biến đổi vĩ mô (ví dụ tái tổ chức toàn bộ lịch của một giáo viên quá tải). Đây không phải lỗi cài đặt mà là *giới hạn bản chất* của tìm kiếm cục bộ với một toán tử — và là động lực trực tiếp cho ALNS ở Mục 6, nơi cơ chế phá hủy quy mô lớn cho phép nhảy ra khỏi cao nguyên.

6 Thuật toán Adaptive Large Neighborhood Search (ALNS)

Tầng Metaheuristic phục vụ ngữ cảnh *quy mô lớn, chạy theo lô*, nơi cần chất lượng cao và sẵn sàng đổi vài giây tính toán. ALNS hoạt động theo chu trình *phá hủy & tái tạo* (ruin & recreate): mỗi vòng phá $\sim 15\%$ số lớp đang xếp rồi tái tạo bằng một cặp toán tử được chọn thích nghi, sau đó quyết định chấp nhận theo Simulated Annealing. Tổng quan ở Thuật toán 2.

6.1 Ba toán tử Phá hủy (Random, Worst-Time, Related-Teacher)

Mỗi toán tử nhắm một dạng bế tắc khác nhau:

- Random: gỡ ngẫu nhiên k lớp — đa dạng hóa, tránh thiên kiến.
- Worst-Time: gỡ k lớp *thời lượng lớn nhất* — giải phóng các khối thời gian công kênh, chiếm nhiều tài nguyên nhất.
- Related-Teacher: chọn giáo viên có lịch dày nhất rồi gỡ cụm lớp của họ — tập trung tháo gỡ điểm xung đột.

Hình 7 minh họa ba toán tử trên cùng một lịch hiện tại (khối = lớp đang xếp; khối tô đỏ nét đứt = lớp bị gỡ).

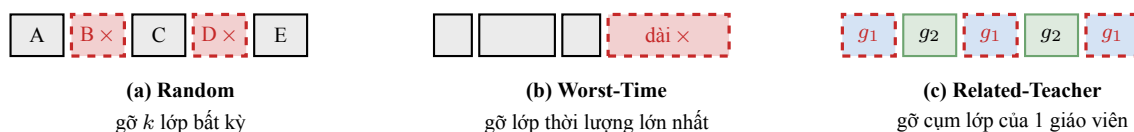


Figure 7: Ba toán tử Phá hủy. Random gỡ ngẫu nhiên; Worst-Time gỡ khối thời lượng dài nhất; Related-Teacher gỡ toàn bộ lớp của giáo viên dày lịch nhất (g_1).

6.2 Ba toán tử Sửa chữa (Greedy, Regret-2, First-Possible)

- Greedy: chèn lại theo số rồi thời lượng giảm dần, mỗi lớp vào vị trí trống đầu tiên.
- Regret-2: ưu tiên lớp “hối tiếc” cao — lớp càng ít phòng khả thi (đếm bằng `feasible_room_count`) thì điểm $1/\text{cnt}$ càng lớn, được chèn trước để không bị kẹt về sau.
- First-Possible: xáo trộn ngẫu nhiên rồi chèn vào chỗ đầu tiên — tạo nhiều cấu trúc.

Ba toán tử khác nhau ở *thứ tự ưu tiên* các lớp chờ khi chèn lại (Hình 8); số trong vòng tròn là thứ tự chèn.

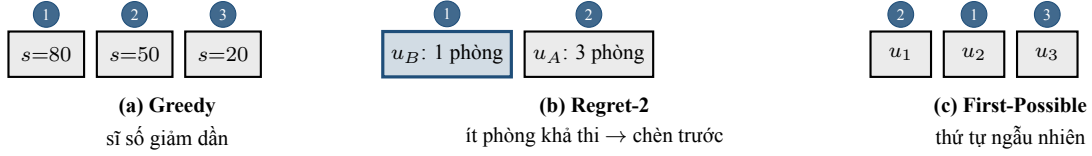


Figure 8: Ba toán tử Sửa chữa khác nhau ở thứ tự chọn lớp chờ để chèn (số trong vòng tròn). Greedy theo sỡ số giảm dần; Regret-2 ưu tiên lớp ít phòng khả thi nhất (u_B); First-Possible theo thứ tự xáo trộn ngẫu nhiên. Cả ba đều đặt mỗi lớp vào vị trí trống đầu tiên tìm được.

6.3 Cập nhật trọng số Adaptive Roulette-Wheel (π_1, π_2, π_3)

Mỗi toán tử mang trọng số w_j (khởi tạo 1); toán tử được bốc bằng bánh xe roulette với xác suất tỉ lệ trọng số:

$$p_j = \frac{w_j}{\sum_{\ell} w_{\ell}}. \quad (9)$$

Sau mỗi vòng, toán tử vừa dùng nhận điểm thưởng π theo mức đóng góp — tạo nghiệm tốt nhất mới ($\pi_1=10$), cải thiện nghiệm hiện hành ($\pi_2=5$), được SA chấp nhận dù không cải thiện ($\pi_3=2$), bị bác (0) — rồi trọng số cập nhật theo trung bình trượt mũ với hệ số học λ :

$$w_j \leftarrow \lambda w_j + (1 - \lambda) \pi. \quad (10)$$

6.4 Tiêu chí chấp nhận bằng Luyện kim mô phỏng (Simulated Annealing)

Để thoát cực trị cục bộ, nghiệm lùi bước vẫn được chấp nhận với xác suất Boltzmann $\exp(-\Delta/T)$, với $\Delta = Q_{\text{best}} - Q_{\text{new}} \geq 0$ và nhiệt độ giảm dần $T \leftarrow \alpha T$ ($T_0=10$, $\alpha=0.98$). Đầu chạy (T lớn) ưu tiên khám phá; cuối chạy ($T \rightarrow 0$) ưu tiên khai thác.

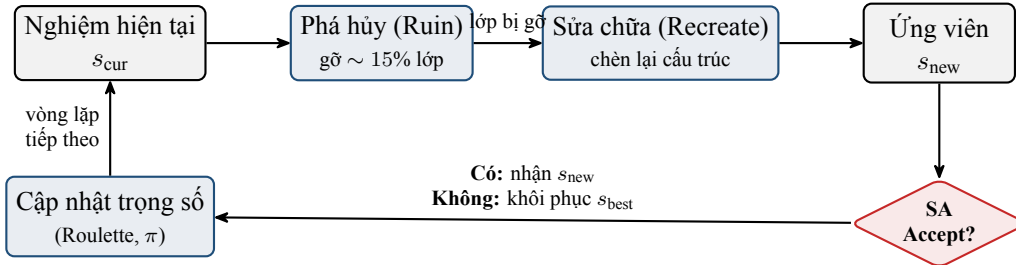


Figure 9: Chu trình Phá hủy & Sửa chữa (Ruin & Recreate) của ALNS kết hợp đánh giá chấp nhận bằng Luyện kim mô phỏng (SA) và cập nhật trọng số toán tử (Adaptive).

Algorithm 2: ALNS = Ruin & Recreate + Adaptive + Simulated Annealing

Input: hạt giống Greedy; $T_0, \alpha, \lambda, (\pi_1, \pi_2, \pi_3)$

```

1  $s_{\text{best}} \leftarrow s_{\text{cur}} \leftarrow \text{Greedy}; T \leftarrow T_0;$ 
2 for mỗi vòng lặp (tới khi hết ngân sách hoặc xếp đủ  $N$  lớp) do
3   bốc cặp (Phá hủy, Sửa chữa) theo roulette (9);
4   phá  $\lceil 0.15 Q \rceil$  lớp; tái tạo  $\Rightarrow s_{\text{new}};$ 
5   if  $Q_{\text{new}} > Q_{\text{best}}$  then
6     | nhận;  $s_{\text{best}} \leftarrow s_{\text{new}}; \pi \leftarrow \pi_1;$ 
7   else if  $Q_{\text{new}} > Q_{\text{cur}}$  then
8     | nhận;  $\pi \leftarrow \pi_2;$ 
9   else if  $\text{rand}() < e^{-\Delta/T}$  then
10    | nhận (lùi bước);  $\pi \leftarrow \pi_3;$ 
11  else
12    | khôi phục  $s_{\text{best}}; \pi \leftarrow 0;$ 
13  cập nhật trọng số (10);  $T \leftarrow \alpha T;$ 
14 return  $s_{\text{best}};$ 

```

7 Phân tích độ nhạy và tinh chỉnh tham số (Parameter Tuning)

Mỗi tham số của ALNS được biện minh bằng đồ thị độ nhạy, không phải chọn tùy ý. Mọi thí nghiệm chạy trên instance nghên *test09* ($N=200, M=5$) — nơi động lực học bộc lộ rõ nhất — lấy trung bình 5 hạt giống ngẫu nhiên.

7.1 Sự tiến hóa của xác suất chọn toán tử (cân bằng Khám phá – Khai thác)

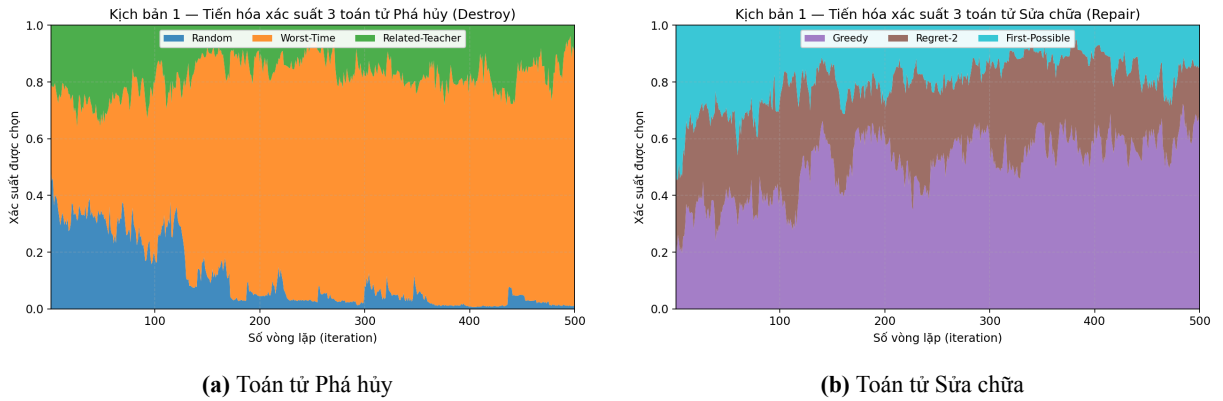


Figure 10: Miền xác suất chọn toán tử theo vòng lặp (tổng mỗi cột = 1), tính theo (9).

Hình 10 cho thấy cơ chế thích nghi *thực sự học*: khởi đầu gần đồng đều (giai đoạn *khám phá*), thuật toán nhanh chóng dồn xác suất cho Worst-Time (tỉ trọng cuối ≈ 0.89) và Greedy bên sửa chữa (≈ 0.69) — giai đoạn *khai thác* chiến lược hiệu quả nhất trên instance nghên phòng, trong khi phá ngẫu nhiên bị đào thải. Đây là minh chứng định lượng cho giá trị của adaptive selection.

7.2 Tác động của hệ số học λ (Learning Rate)

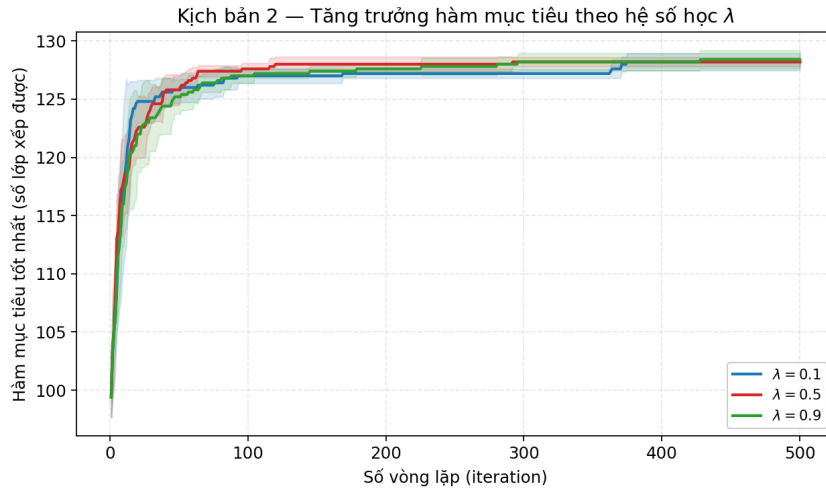


Figure 11: Tăng trưởng hàm mục tiêu theo $\lambda \in \{0.1, 0.5, 0.9\}$; dải mờ là ± 1 độ lệch chuẩn trên 5 hạt giống.

Ba giá trị cho chất lượng trung bình gần như nhau (≈ 128 lớp), nên tiêu chí phân định là *độ ổn định*: $\lambda=0.5$ cho độ lệch chuẩn cuối kỳ ($\sigma \approx 0.40$) nhỏ hơn $\sim 2\times$ so với $\lambda=0.1$ (quên nhanh, $\sigma \approx 0.75$) và $\lambda=0.9$ ($\sigma \approx 0.80$). Ta chọn $\lambda = 0.5$ vì hành vi tái lập và đáng tin cậy nhất.

7.3 Sự biến thiên Nhiệt độ T và hệ số làm mát α (Cooling Schedule)

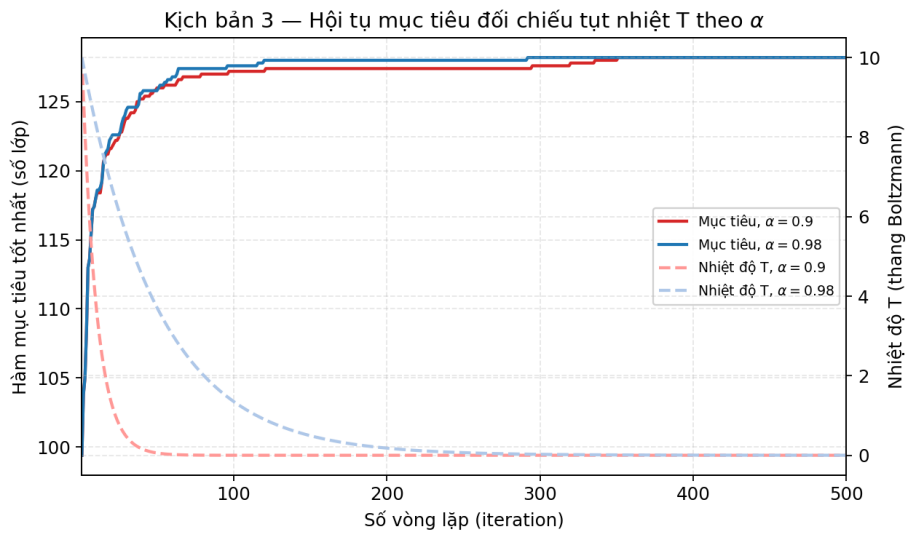


Figure 12: Đồ thị kép: hội tụ mục tiêu (đường liền, trục trái) đối chiếu tụt nhiệt T (đường đứt, trục phải) cho $\alpha = 0.90$ và $\alpha = 0.98$.

Với $\alpha=0.90$, nhiệt độ rơi về ≈ 0.06 chỉ sau ~ 50 vòng — sau đó $e^{-\Delta/T} \approx 0$, thuật toán *đóng băng* thành leo đồi thuần túy, dễ chết yểu ở cực trị cục bộ. Với $\alpha=0.98$, T còn ≈ 3.7 tại vòng 50 và chỉ tiệm cận 0 quanh vòng 250, giữ “van xả” khám phá lâu hơn mà gần như không tốn thêm chi phí. Ta chọn $T_0 = 10, \alpha = 0.98$.

8 Kết quả thực nghiệm

8.1 Thiết lập đánh giá công bằng (Môi trường, Time-limit, Seeds)

Để so sánh công bằng, mọi thuật toán được đặt trên cùng một sân chơi:

- **Ngân sách thời gian chung:** mỗi instance, LS, ALNS và CP-SAT cùng nhận *một* time-limit 30s. Giá trị này xuất phát từ chi phí thực đo của ALNS (toán tử Regret-2 $\approx 0.5s/vòng$ trên $N=500$, cần $\sim 50-100$ vòng để phát huy); 30s cũng phù hợp ngữ cảnh xếp lịch theo lô. Để không lãng phí ngân sách trên instance dễ, ALNS *dừng sớm* khi đã xếp đủ N lớp. Ngân sách là “mềm” (kiểm tra giữa các vòng): một vòng Regret-2 trên N lớn có thể kéo dài vài giây nên ALNS đôi khi vượt nhẹ 30s (tới $\sim 37s$) — điều này chỉ *có lợi* cho ALNS, vậy mà nó vẫn thua LS trên instance lớn nghẽn.
- **Seed cố định:** các thuật toán ngẫu nhiên (LS, ALNS) lập với bộ seed cố định lấy từ $\{1, 7, 13, 21, 42\}$ — 3 lần (nhóm nhỏ/vừa) và 2 lần (nhóm lớn, do chi phí cao) — báo cáo (min, max, avg, std) để tái lập.
- **Cô lập:** mỗi lần chạy nạp lại instance sạch; Greedy là hạt giống chung. Ngoài ra ta ghi thêm cột *LS (hội tụ)* — LS dừng tự nhiên — để phản ánh đặc tính phản hồi nhanh của tầng Heuristic.

Table 3: Môi trường thực nghiệm và giao thức đánh giá công bằng.

Hạng mục	Giá trị
Hệ điều hành	macOS-26.3.1-arm64-arm-64bit
CPU	arm (11 nhân)
Python	3.10.20
OR-Tools	9.15.6755
Ngân sách thời gian chung	30 giây / instance (LS, ALNS, CP-SAT)
Số lần lặp (seed)	3 (nhỏ/vừa), 2 (lớn)
Tập seed	lấy từ $\{1, 7, 13, 21, 42\}$

8.2 Bảng kết quả đầy đủ (phân nhóm: Nhỏ, Trung bình, Lớn)

Ba nhóm dữ liệu bao phủ phổ quy mô và độ khó. Nhóm nhỏ có chuẩn CP-SAT tối ưu (Bảng 4); nhóm trung bình và lớn trộn instance dư tài nguyên với instance nghẽn (Bảng 5, 6).

Table 4: Nhóm nhỏ ($N \leq 30$): CP-SAT là chuẩn tối ưu. LS/ALNS trung bình các seed.

Instance	N	M	CP-SAT	Greedy	LS avg	ALNS avg	%opt(LS)	%opt(ALNS)
test01	5	3	5	5	5.0	5.0	100%	100%
test03	20	5	20	20	20.0	20.0	100%	100%
hustack05	20	3	20	20	20.0	20.0	100%	100%
test02	10	5	10	10	10.0	10.0	100%	100%
hustack08	10	2	9	9	9.0	9.0	100%	100%

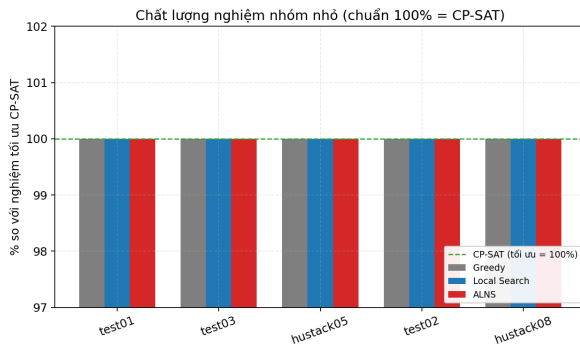
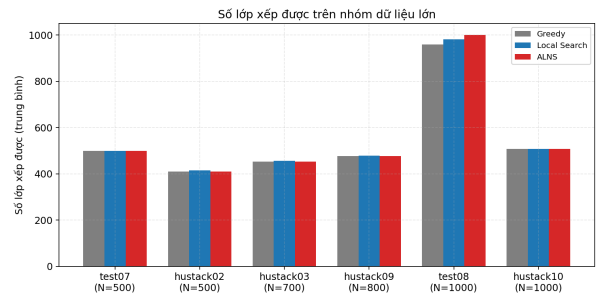
Table 5: Nhóm trung bình ($N \approx 100\text{--}200$): trộn instance dư tài nguyên và nghẽn phòng (M nhỏ). Cùng ngân sách 30s.

Instance	N	M	Greedy	LS	ALNS		CP-SAT	$\Delta(\text{ALNS} - \text{Gr})$
				min/avg/max	min/avg/max	std		
test05	100	15	100	100/100.0/100	100/100.0/100	0.00	100 (OPTI)	+0.0
comp11	162	5	147	162/162.0/162	162/162.0/162	0.00	162 (OPTI)	+15.0
test09	200	5	125	127/127.0/127	127/128.3/129	0.94	131 (OPTI)	+3.3
hustack01	200	10	182	184/184.3/185	193/193.7/194	0.47	196 (FEAS)	+11.7

Table 6: Nhóm lớn ($N \geq 500$). Cùng ngân sách 30s; LS/ALNS trung bình các seed.

Instance	N	M	Greedy	LS	ALNS		CP-SAT	$\Delta(\text{ALNS} - \text{Gr})$
				min/avg/max	min/avg/max	std		
test07	500	30	499	500/500.0/500	500/500.0/500	0.00	500 (OPTI)	+1.0
hustack02	500	50	410	415/415.5/416	410/410.5/411	0.50	410 (FEAS)	+0.5
hustack03	700	50	453	455/455.5/456	453/453.0/453	0.00	453 (FEAS)	+0.0
hustack09	800	50	477	478/478.0/478	477/477.0/477	0.00	478 (FEAS)	+0.0
test08	1000	50	959	982/982.5/983	1000/1000.0/1000	0.00	990 (FEAS)	+41.0
hustack10	1000	50	508	508/508.0/508	508/508.0/508	0.00	508 (OPTI)	+0.0

8.3 Biểu đồ so sánh (chất lượng nghiệm và thời gian chạy)

**(a)** Nhóm nhỏ: % so với CP-SAT**(b)** Nhóm lớn: số lóp xếp được**Figure 13:** Chất lượng nghiệm: nhóm nhỏ bám chuẩn tối ưu; nhóm lớn — ALNS đạt tối ưu trên test08 (còn khe cấu trúc lớn), còn trên các instance nghẽn HUSTack thì LS nhỉnh hơn.

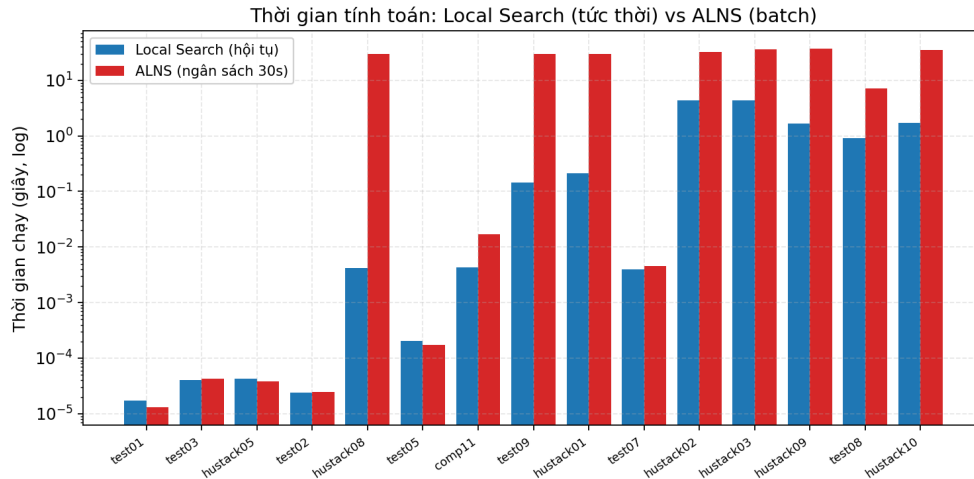


Figure 14: Thời gian tính toán (thang log): Local Search hội tụ rất nhanh — mili-giây trên dữ liệu nhỏ/thưa, tới vài giây trên instance lớn nghèn (vd hustack02 $\approx$ 4.4s) — vẫn nhanh hơn ALNS nhiều bậc; ALNS dùng trọn ngân sách 30s trên instance nghèn.

8.4 Phân tích và thảo luận

Giao thức công bằng (cùng ngân sách 30s) cho ra bốn nhận định — trong đó nhận định thứ ba là một phát hiện đáng giá mà chỉ phép so sánh *đồng thời thời gian* mới bộc lộ.

- Quy mô nhỏ: heuristic là đủ.** Trên toàn nhóm nhỏ, Greedy, LS, ALNS đều đạt 100% tối ưu CP-SAT (Bảng 4). Thuật toán phức tạp ở đây là dư thừa.
- Thuật toán Tham lam (Greedy best-fit) thu hẹp mạnh khoảng cách, nhưng vẫn còn dư địa.** Nhờ chiến lược sắp xếp ưu tiên (thời lượng tăng, sĩ số tăng) kết hợp với best-fit phòng (Mục 3), bản thân Greedy đã tiệm cận rất sát mức tối ưu trên các tập dữ liệu nghèn lớn — thậm chí đạt tối ưu tuyệt đối trên hustack10 (508). Dù vậy, thuật toán vẫn để sót một số lượng nhỏ các lớp (test09: 125, test08: 959, comp11: 147). Phân hao hụt này chính là không gian tối ưu dành cho các thuật toán LS/ALNS thể hiện.
- Điểm giao thoa hiệu năng giữa LS và ALNS phụ thuộc vào đặc tính của dữ địa cải thiện, không thuần túy do quy mô.**
 - Khi hệ thống tồn tại các *khoảng trống cấu trúc lớn* cần được tái sắp xếp, ALNS hoàn toàn áp đảo: trên hustack01, ALNS đạt 193.7 so với LS 184.3; trên test08 ($N=1000$), ALNS vượt tới mức tối ưu tuyệt đối 1000 trong khi LS mắc kẹt ở “hiệu ứng cao nguyên” với 982.5. Cơ chế phá vỡ và tái tạo (ruin & recreate) đã phát huy đúng kỳ vọng.
 - Ngược lại, trên các tập dữ liệu thực tế có độ nghèn cực cao của HUSTack, LS lại chiến thắng: hustack02 (415.5 vs 410.5), hustack03 (455.5 vs 453.0), hustack09 (478.0 vs 477.0).

Nguyên nhân cốt lõi nằm ở *chi phí tính toán của mỗi vòng lặp*. Toán tử Regret-2 của ALNS có độ phức tạp quét (lớp $\times$ phòng $\times$ tiết) nên chi phí tăng vọt theo N , khiến ALNS thực hiện được rất ít vòng lặp trong ngân sách 30s. Trong khi đó, mỗi vòng lặp của LS tốn chi phí cực rẻ, tạo ra sự áp đảo về “số vòng khám phá”. **Bài học rút ra:** dưới ràng buộc thời gian thực tế (time-limit), tốc độ thực thi của vòng lặp quan trọng ngang bằng với chất lượng của vòng lặp đó; không một phương pháp xấp xỉ nào là thống trị tuyệt đối — điều này càng củng cố giá trị của kiến trúc hệ thống đa tầng có định tuyến.

- Bộ giải chính xác không thống trị trong ngân sách.** Bức tranh trộn lẫn: CP-SAT đạt tối ưu trên test09 (131), hustack10 (508) và thậm chí *dẫn đầu* trên hustack01 (196, hơn mọi heuristic); nhưng trên hustack02/hustack03 nó *kẹt ngay mức Greedy* (410, 453) và thua LS, còn trên test08 chỉ đạt feasible 990 — kém ALNS (1000). Đây là minh chứng cho thiết kế đa tầng: ở quy mô nghèn lớn,

heuristic/metaheuristic trong cùng thời gian có thể cho nghiệm *tốt hơn* bộ giải “tối ưu”. Đánh đổi thời gian–chất lượng được định lượng ở Bảng 7.

Table 7: Đánh đổi thời gian–chất lượng: Local Search (hội tụ tự nhiên) so với ALNS (ngân sách 30s). $\Delta\%$ là chênh lệch số lớp của ALNS so với LS (giá trị *âm* nghĩa là LS tốt hơn, ví dụ hustack02).

Instance	Nhóm	N	LS (ms)	ALNS (s)	LS avg	ALNS avg	$\Delta\%$
test01	small	5	0.0	0.00	5.0	5.0	+0.00%
test03	small	20	0.0	0.00	20.0	20.0	+0.00%
hustack05	small	20	0.0	0.00	20.0	20.0	+0.00%
test02	small	10	0.0	0.00	10.0	10.0	+0.00%
hustack08	small	10	4.2	30.00	9.0	9.0	+0.00%
test05	medium	100	0.2	0.00	100.0	100.0	+0.00%
comp11	medium	162	4.3	0.02	162.0	162.0	+0.00%
test09	medium	200	144.0	30.03	127.0	128.3	+1.05%
hustack01	medium	200	212.6	30.02	184.3	193.7	+5.06%
test07	large	500	4.0	0.00	500.0	500.0	+0.00%
hustack02	large	500	4401.1	32.77	415.5	410.5	-1.20%
hustack03	large	700	4414.5	36.12	455.5	453.0	-0.55%
hustack09	large	800	1677.3	36.98	478.0	477.0	-0.21%
test08	large	1000	908.3	7.08	982.5	1000.0	+1.78%
hustack10	large	1000	1693.8	34.87	508.0	508.0	+0.00%

9 Kết luận

Báo cáo đã xây dựng và đánh giá công bằng một hệ giải *đa tầng* (Exact $\rightarrow$ Heuristic $\rightarrow$ Metaheuristic) cho bài toán xếp thời khóa biểu, với các kết luận chính:

- (1) **Định tuyến theo độ chặt tài nguyên là quyết định đúng.** Thực nghiệm cho thấy độ khó nằm ở mức cạnh tranh tài nguyên chứ không ở N : CP-SAT giải tối ưu dữ liệu thừa tới $N=600$ nhưng chỉ đạt feasible (còn gap) trên dữ liệu thực nghiệm ngay từ $N=200$. Đây là nền tảng cho kiến trúc.
- (2) **Mỗi tầng phục vụ đúng một ngữ cảnh.** CP-SAT cho chuẩn tối ưu ở quy mô nhỏ; Local Search cho phản hồi tức thời (mili-giây); ALNS thoát được cao nguyên mà LS mắc kẹt trên nhiều instance nghẽn (hustack01, test09). Tuy nhiên, phép so sánh *đồng thời gian* cho thấy điểm giao phụ thuộc *loại khe cái thiện*: ALNS thắng khi còn khe cấu trúc lớn (hustack01; test08 đạt tối ưu 1000 vs LS 982.5), nhưng trên instance lớn nghẽn HUSack, LS nhiều-vòng-rẽ vượt ALNS (hustack02: 415.5 vs 410.5) do chi phí Regret-2 tăng theo N . CP-SAT cũng không thống trị: dẫn đầu ở hustack01 (196) nhưng kẹt mức Greedy ở hustack02/03. Greedy là hạt giống chung gắn kết tất cả.
- (3) **Tham số có cơ sở.** $\lambda=0.5$ và $\alpha=0.98$ được chọn từ đồ thị độ nhạy với tiêu chí kỹ thuật (độ ổn định, giữ “van xả” khám phá), thay vì cảm tính.

Khuyến nghị triển khai.

Module	Dùng khi	Vì sao
CP-SAT	Quy mô khoa/viện ($N \lesssim 100$), cần đảm bảo tối ưu	Nghiem tối ưu có chứng nhận; warm-start giúp nhanh và an toàn.
Local Search	Ứng dụng tương tác (chỉnh tay, kéo-thả)	Độ trễ thấp (mili-giây tới vài giây); bảo đảm không làm xấu nghiệm.
ALNS	Xếp lịch toàn trường, chạy theo lô	Vượt cao nguyên, nhồi thêm lớp ở điểm nghẽn; đổi thời gian lấy chất lượng.

Bài học. Giá trị cốt lõi không nằm ở một thuật toán đơn lẻ mà ở việc *chọn đúng công cụ cho đúng tình huống* và lập luận được cho từng lựa chọn bằng số liệu. Toàn bộ kết quả/đồ thị được sinh tự động từ chính mã nguồn được phân tích, đảm bảo tái lập.

Tài liệu tham khảo

1. L. Perron, F. Didier. *CP-SAT Solver, Google OR-Tools*. Google, 2024. https://developers.google.com/optimization/cp/cp_solver
2. S. Ropke, D. Pisinger. “An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows.” *Transportation Science*, 40(4): 455–472, 2006.
3. D. Pisinger, S. Ropke. “Large Neighborhood Search.” In: *Handbook of Metaheuristics*, 2nd ed., Springer, 399–419, 2010.
4. P. Shaw. “Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems.” In: *Principles and Practice of Constraint Programming (CP’98)*, LNCS 1520, 417–431, 1998.
5. S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. “Optimization by Simulated Annealing.” *Science*, 220(4598): 671–680, 1983.
6. F. Glover. “Ejection chains, reference structures and alternating path methods for traveling salesman problems.” *Discrete Applied Mathematics*, 65(1–3): 223–253, 1996.
7. A. Schaerf. “A Survey of Automated Timetabling.” *Artificial Intelligence Review*, 13(2): 87–127, 1999.
8. B. McCollum et al. “Setting the Research Agenda in Automated Timetabling: The Second International Timetabling Competition (ITC-2007).” *INFORMS Journal on Computing*, 22(1): 120–130, 2010.